

LAB 1 • PREPARATION

Software Engineering

Introduction, key concepts, software costs & maintenance, and challenges

Truong Phat Tu

Master of Computer Science in Concordia University 2026-2028



Topics in this lab

- 1 Introduction to Software Engineering
- 2 Key Concepts in Software Engineering
- 3 Software Cost and Maintenance
- 4 Challenges in Software Engineering
- 5 Reference



1 · INTRODUCTION

What is Software Engineering?

Engineering is ... the application of scientific principles and methods to the construction of useful structures & machines.

EXAMPLES

Mechanical engineering

Civil engineering

Electrical engineering

Nuclear engineering

Chemical engineering

Aeronautical engineering



Historical context: the NATO Conferences

The term is **54 years old**: NATO Conferences

Garmisch, Germany

October 7-11, 1968

Rome, Italy

October 27-31, 1969

The reality is finally beginning to arrive.
Computer science as the scientific basis.



Software engineering in professional development

- Development of software systems whose size/complexity warrants **teams of engineers**
- Multi-person construction
- The economies of all developed nations are dependent on software
- Concerned with theories, methods and tools for professional software development
- Expenditure represents a significant fraction of GNP in all developed countries



Architect

Developer

Manager

Tester

Customer

User

Complex systems · indefinite lifespan · numerous stakeholders · system families · reuse to amortize cost · maintenance cost over 60%

Software engineering vs. software programming

Software programming

SWP

- Single developer, 'toy' applications
- Short lifespan
- Single or few stakeholders: Architect = Developer = Manager = Tester = Customer = User
- One-of-a-kind systems
- Built from scratch
- Minimal maintenance

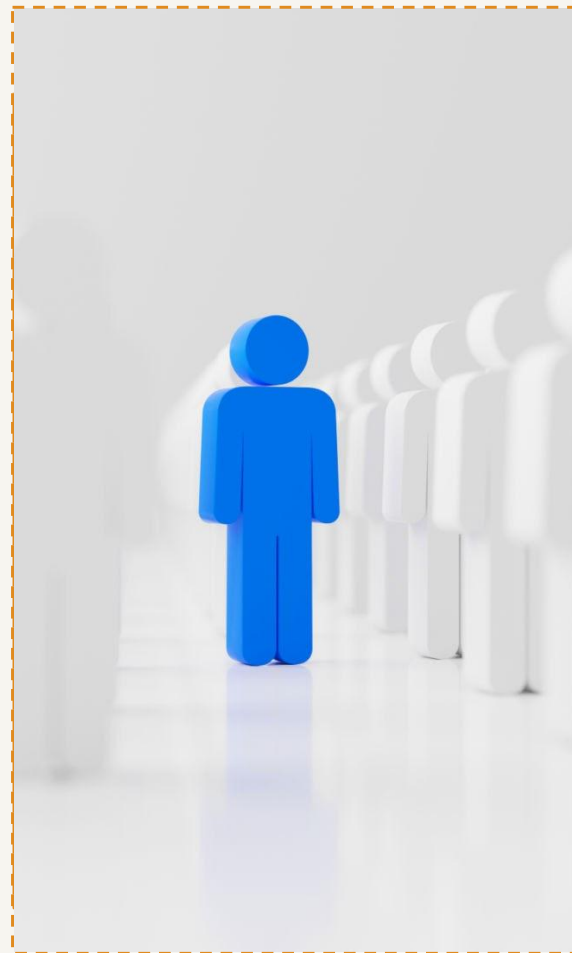
Software engineering

SWE

- Teams of developers with multiple roles
- Complex systems, indefinite lifespan
- Numerous stakeholders: Architect ≠ Developer ≠ Manager ≠ Tester ≠ Customer ≠ User
- System families
- Reuse to amortize costs
- Maintenance accounts for over 60% of overall development costs

SE is a unique brand of engineering

- Software is malleable
- Software construction is human-intensive
- Software is intangible
- Software problems are unprecedentedly complex
- Software directly depends upon the hardware (top of the system engineering 'food chain')
- Software solutions require unusual rigor
- Software has discontinuous operational nature



Understanding software costs

Development

+

Maintenance (evolution)

=

Software production

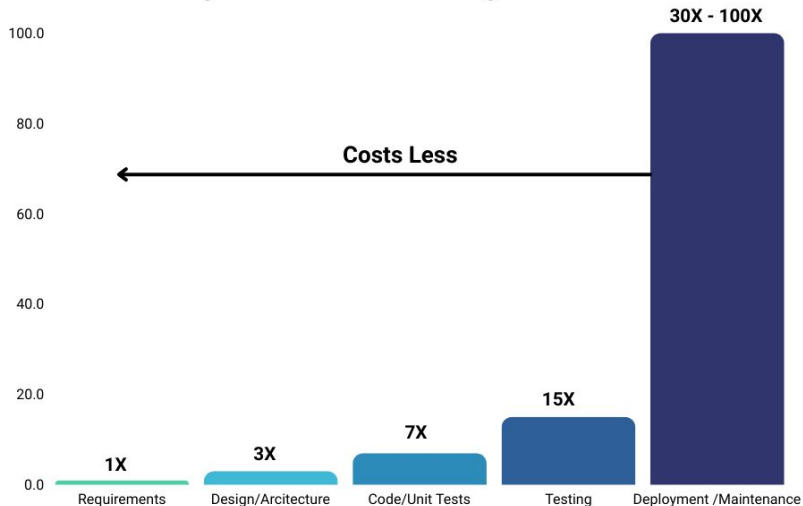
Quicker development is not always preferable

- Higher up-front costs may defray downstream costs
- Poorly designed/implemented software is a critical cost factor

SUGGESTED

Maintenance can outweigh initial development because software lives on, adapting to new environments and user requests.

Average Cost of Software Bugs in SDLC



Source: National Institute of Standards & Technology, US

<https://www.nist.gov/system/files/documents/director/planning/report02-3.pdf>

Breakdown of maintenance costs

Corrective

Reactive changes after delivery to correct discovered problems (faults).

Adaptive

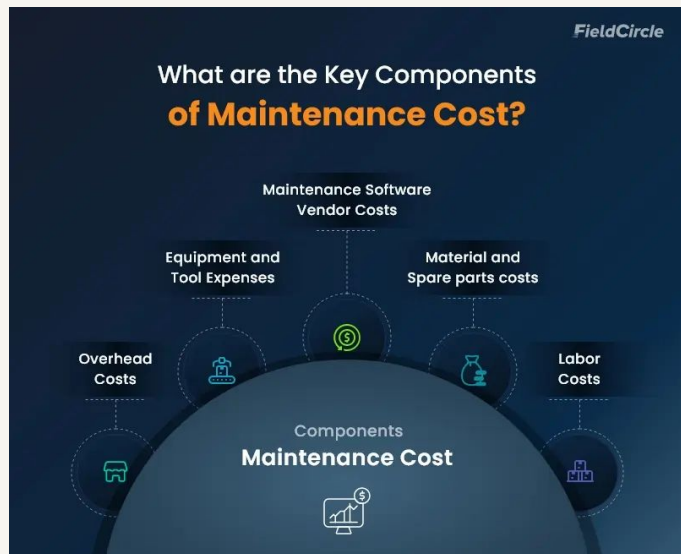
Changes that keep the software usable as its environment changes (OS, hardware, platform).

Perfective

Changes driven by user requests: new or extended functions, better performance, easier use.

SUGGESTED

Lientz & Swanson defined these three categories; some later sources add a fourth, preventive maintenance.



Economic and management aspects: managing resources



Human resources

- Team size and roles
- Skill levels
- Effort estimation



Time

- Schedule and deadlines
- Milestones



Technical resources

- Development tools
- Hardware requirements
- Software licenses



Risk factors

- Scope changes
- Technical challenges



Why software engineering is different

Ever-present difficulties: the complexity of software, human-intensive processes, and software malleability.

SUGGESTED

Brooks' essential difficulties (1987)

Complexity

No two parts are alike, so software is complex for its size.

Conformity

It must conform to many human institutions and systems.

Changeability

Constant pressure for change.

Invisibility

Software has no natural physical form to picture.



Estimating software development resources



Expert judgment

Estimates drawn from the experience of skilled people.



Analogy-based

Estimates by comparison with similar past projects.



Algorithmic cost modeling

Formula-based models that estimate effort, cost and schedule.

Challenge: predicting time, effort, and budget, and how maintenance costs compare to development costs.

SUGGESTED

Example: COCOMO (Boehm, 1981), fitted on data from 63 TRW projects.



**COCOMO
Model
in
Software
Engineering**

SOFTWARE DEVELOPMENT CONCEPTS



**Need
identification**



**Planning
& Designing**



Implementation



Testing



Deployment



**Functionality
testing**



**User experience
testing**



**Performance
& load testing**



**Compatibility
testing**



Security testing



**Agile
development**



**Waterfall
development**



Spiral development



**Offshore
development**



Maintenance

References

- [1] Naur, P., & Randell, B. (Eds.). (1969). *Software Engineering: Report on a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7th-11th October 1968*. Brussels: NATO Scientific Affairs Division. <http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>
-
- [2] Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *IEEE Computer*, 20(4), 10-19.
-
- [3] Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall.
-
- [4] Lientz, B. P., & Swanson, E. B. (1980). *Software Maintenance Management: A Study of the Maintenance of Computer Application Software in 487 Data Processing Organizations*.
-
- [5] Course lecture notes: preparation_lab1.txt (Parts 1-4).
-